Table of Contents

3.2.1 | OpenSees Analysis

This section analyzes the period of the tree - tree fort system. The OpenSees model is schematically shown below:

```
    y
    ^
    |
m2  o  Node 3 (Top mass)
    |
k2  |   Spring 2 (Between m1 and m2)
    |
m1  o  Node 2 (Middle mass)
    |
k1  |   Spring 1 (Between base and m1)
    |
    o  Node 1 (Fixed Base)
-----------
(Ground)
```

3.2.2 | osp-mod1 Model values

variable	value	[value]	description
mass1	1.5	1.5	mass of tree fort, kN/g
mass2	3.5	3.5	mass of branches, kN/g
trk1	1100	1100	lower tree trunk stiffness, kN/cm
trk2	2100	2100	upper tree trunk stiffness, kN/cm

3.2.3 | Insert ops-mod1 Output

Model Input

```python
import openseespy.opensees as ops
import numpy as np
import math
import matplotlib.pyplot as plt
import opsvis as opsv
# ------------------------------------------------------------------------
# 1. Model Initialization
# ------------------------------------------------------------------------
ops.wipe()
ops.model("BasicBuilder", "-ndm", 2, "-ndf", 2)

# ------------------------------------------------------------------------
# 2. Define Parameters
# ------------------------------------------------------------------------
# Mass values (e.g., in tons)
m1 = 1.5
```

```
m2 = 3.5

# Stiffness values (e.g., in kN/m)
k1 = 1100.0
k2 = 2100.0

# ------------------------------------------------------------------
# 3. Create Nodes
# ------------------------------------------------------------------
# Base node (fixed)
ops.node(1, 0.0, 0.0)

# Node 1
ops.node(2, 0.0, 2.0)

# Node 2
ops.node(3, 0.0, 4.0)

# Fix the base node in both X and Y directions
ops.fix(1, 1, 1)
# Restrain vertical (Y) displacement and rotation at mass nodes
# to represent a pure 2D shear/lateral lollipop system
ops.fix(2, 0, 1)  # Free in X, Fixed in Y
ops.fix(3, 0, 1)  # Free in X, Fixed in Y
# ------------------------------------------------------------------
# 4. Assign Masses
# ------------------------------------------------------------------

ops.mass(2, m1, 0.0)
ops.mass(3, m2, 0.0)

# ------------------------------------------------------------------
# 5. Define Elements
# ------------------------------------------------------------------
# Use elastic truss elements to represent the lateral springs
# Assign high axial stiffness (E * A) and 0.0 length
EA = 1e8

# Spring 1 between Node 1 and Node 2
ops.uniaxialMaterial("Elastic", 1, 1100.0)
ops.element("Truss", 1, 1, 2, EA, 1)

# Spring 2 between Node 2 and Node 3
ops.uniaxialMaterial("Elastic", 2, 2100.0)
ops.element("Truss", 2, 2, 3, EA, 2)

# ------------------------------------------------------------------
# 6. Eigenvalue Analysis & Results Output
# ------------------------------------------------------------------
num_modes = 1
eigenvalues = ops.eigen(num_modes)

periods = []
frequencies = []
```

```
outL = []
for i in range(num_modes):
    lamb = eigenvalues[i]
    omega = math.sqrt(lamb)
    freq = omega / (2 * math.pi)
    period = 2 * math.pi / omega
    periods.append(period)
    frequencies.append(freq)
    resS = f'Mode {i + 1}: Period = {period:.4f} s | Frequency = {freq:.4f} Hz'
    outL.append(resS)
outS = chr(10).join(item for item in outL)
with open("output.txt", 'w') as f1:
    f1.write(outS)
print("text output written")
# ------------------------------------------------------------------------
# 7. Model Visualization
# ------------------------------------------------------------------------

# Plot the defined model to check geometry (this is an axes mdoel)
mod1 = opsv.plot_model()
fig1 = mod1.get_figure()
plt.title("2-DOF Lollipop Model Geometry")
fig1.savefig("figure1.png", dpi=200)
print("figure 1 written")
plt.show(block=False)

# Plot the mode shape for the first mode (this is a figure)
fig2 = opsv.plot_mode_shape(1, 2.0)  # Scaling factor of 2.0 for visibility
plt.title("Mode Shape 1")
plt.savefig("figure2.png", dpi=200)
print("figure 2 written")
plt.show()
```

Model Plots

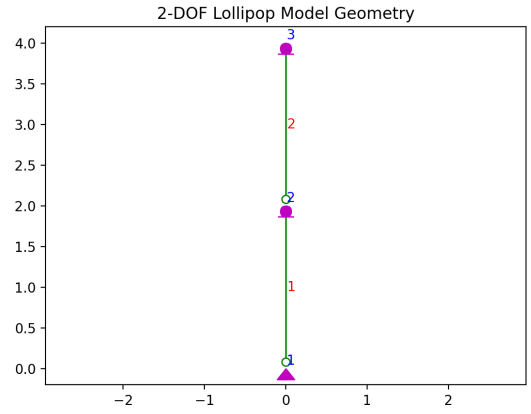

Fig. 1 - OPS Model

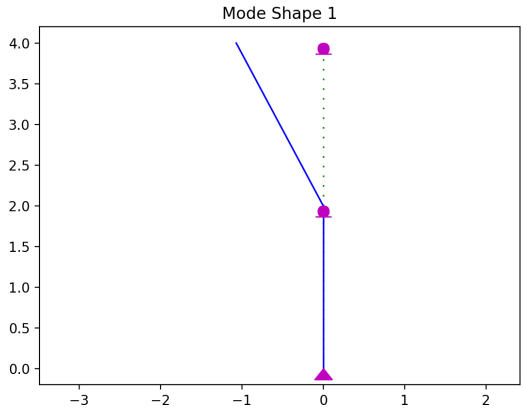

Fig. 2 - OPS First Mode

Model Text

```
Mode 1: Period = 11.7548 s | Frequency = 0.0851 Hz
```